Day 1 Lab — OAuth 2.0 Flows in Go

Authorization Code + PKCE · Client Credentials · Refresh Token · Device Authorization

Goal of This Lab

By the end of this lab you should be able to, from memory:

- Name the three actors in an OAuth exchange and the endpoints each one talks to.
- Follow an Authorization Code + PKCE flow and say what travels through the browser versus what happens back-channel.
- Explain why `state`, `redirect_uri`, `code_challenge`, and `code_verifier` exist.
- Choose the correct flow for a scenario: Authorization Code, Client Credentials, Refresh Token, or Device Authorization.

You do not need to write code in this lab. You run one Go application, drive it from the browser, and explain what each request is doing.

The Application You Are Running

One Go program starts **three separate servers**, so the network boundaries you would have in production are visible on different ports:

Actor	URL	Role
Client Application	`http://localhost:8082`	The app acting on behalf of the user or itself. **Start here.**
Authorization Server	`http://localhost:8080`	Issues tokens; hosts `/authorize`, `/token`, `/introspect`, `/device_authorization`, `/.well-known/*`, `/jwks`.

Actor	URL	Role
Resource Server (API)	`http://localhost:8081`	Protected API; validates tokens by introspection.

Keeping all three in one process lets you trace a full flow end-to-end. In production, each would be a separate service owned by a different team.

Setup

Requires Go 1.21+ (this demo is pinned to Go 1.25). No Docker, no external services — everything runs in memory, and all dependencies are vendored.

```
cd resource/labs/day1-go-oauth-demo
go run .
```

Watch the terminal. Every request is logged with the actor that handled it, for example:

```
[ClientApp (8082)] GET /start
[AuthServer (8080)] GET /authorize
[ResourceAPI (8081)] GET /api/profile
```

Then open **http://localhost:8082** in your browser.

> Keep the terminal visible next to the browser. The log lines are the point of the lab — they show which actor is talking to which endpoint at each step.

To start any exercise from a clean slate, use the **Reset** action (or visit `http://localhost:8082/admin/reset`). It clears sessions and stored grants.

Exercise 1 — Authorization Code + PKCE

Scenario: A public web app (RepoBoard, client `repoboard-web`, no client secret) wants to read the signed-in user's profile-like resource.

Steps

1. Click **Start Auth Code + PKCE flow**. The client generates a `state` (CSRF protection), a secret `code_verifier`, and its SHA-256 hash `code_challenge`.
2. The browser is redirected from the client (`:8082`) to the Authorization Server (`:8080`). The redirect URL carries `response_type=code`,

`client_id`, `scope`, `state`, and `code_challenge`. The `code_verifier` never leaves the client.
3. Approve the login/consent screen (as user "Alya").
4. The Authorization Server redirects back to `:8082/callback` with a short-lived, single-use `code` and the original `state`.
5. The client verifies `state`, then makes a **back-channel** `POST /token` with the `code` and the `code_verifier`. The server re-hashes the verifier and compares it to the original challenge before issuing tokens.
6. The UI shows the token response: an `access_token`, an `id_token` (because `openid` was requested), and a `refresh_token` (because `offline_access` was requested).
7. Click **Call resource API** — the client sends `Authorization: Bearer <access_token>` to `:8081/api/profile`, which introspects the token and returns the profile.

What to observe

- In the terminal, note the boundary: the code arrives in the **browser** (front-channel), but the token is fetched **server-to-server** (back-channel).
- Look at the callback URL in the address bar. The `code` is visible there. The `code_verifier` is not — it stayed in the client session.

Questions

- What would break if you removed `code_challenge` from the authorization request?
- An attacker intercepts the `code` from the redirect URL. Why can't they exchange it for a token? What exactly stops them?
- Why must the client check that the returned `state` matches the one it sent?

Exercise 2 — Client Credentials

Scenario: A backend worker (Order Service, client `order-service`, confidential) calls an internal Fraud API. There is no end user.

Steps

1. Click **Run Client Credentials demo**. There is **no browser redirect**. The service posts directly to `:8080/token` with `grant_type=client_credentials`, authenticating with HTTP Basic (`client_id` + `client_secret`).
2. The server returns an `access_token` scoped to `fraud.check`. Note there is **no** `refresh_token` and **no** `id_token` — there is no user session to refresh or identify.

3. The client immediately calls `:8081/api/fraud-report` with the token.

Questions

- Why is there no refresh token here? What would a refresh token even mean without a user?
- How does the subject (`sub`) of this token differ from the one in Exercise 1?
- Why does this flow require a client secret when the RepoBoard client did not?

Exercise 3 — Refresh Token

Scenario: The access token from Exercise 1 has expired, and the client wants a fresh one without sending the user through login again.

Steps

1. **Prerequisite:** complete Exercise 1 first so your session holds a `refresh_token`.
2. Click **Use refresh token**. The client posts `grant_type=refresh_token` with the stored refresh token to `:8080/token`.
3. The UI shows a **new** `access_token` and a **new** `refresh_token`. This is refresh-token rotation: the old refresh token is invalidated immediately.

What to observe

- Click **Use refresh token** twice in a row using the *same* old token value. The second attempt fails — rotation means each refresh token is single-use.

Questions

- Why is refresh-token rotation safer than reusing the same long-lived token?
- This server only issues refresh tokens when `offline_access` is in scope. Why gate it behind an explicit scope instead of always issuing one?
- What should a client do when its refresh token is rejected?

Exercise 4 — Device Authorization

Scenario: A CLI tool (Deploy CLI, client `deploy-cli`) needs a user to sign in, but it has no browser or convenient keyboard.

Steps

1. Click **Start Device Authorization demo** (this represents the CLI). The CLI posts to `:8080/device_authorization`.
2. The response contains a long secret `device_code`, a short `user_code`, and a `verification_uri`.
3. Click **Open verification page** (simulating the user on their phone). Enter the `user_code` at `:8080/device`, then approve.
4. Meanwhile the CLI polls `:8080/token` with `grant_type=urn:ietf:params:oauth:grant-type:device_` **Before** you approve, click **Poll token endpoint** — you get `authorization_pending`.
5. **After** approving, click **Poll token endpoint** again — now the token is issued.
6. The CLI calls `:8081/api/deploy` with the new token.

Questions

- Why is the flow split into two codes (`device_code` and `user_code`)? Who sees each one?
- The CLI never handles the user's credentials. Why is that a security improvement?
- What is the purpose of the polling interval and the `slow_down` response?

How the Resource Server Validates Tokens

Every protected endpoint on `:8081` does the same four things:

1. Read the `Authorization: Bearer <token>` header.
2. `POST` the token to the Authorization Server's `/introspect` endpoint.
3. Check `active: true` and that the required scope is present.
4. Return the response, or `401` if validation fails.

The Resource Server does **not** share memory with the Authorization Server — it validates over a standard endpoint, exactly as a separate service would.

> **Looking ahead to Day 2:** here the API asks the Authorization Server to validate an opaque token (introspection). On Day 2 we look at JWT access tokens, where the API can validate the token **locally** by verifying its signature against the server's JWKS — no round trip.

Wrap-up Discussion

- Map each of the four flows to a real system you work with. Which flow does each use, and why?

- In which flows does a token or code travel through the browser? What is the risk, and what control reduces it?
- Which flows involve a human, and which are machine-to-machine? How does that change what tokens are issued?
- If a client secret leaked, which of these flows would be affected, and how badly?